

The Implementation of Linear Regression and Greedy Algorithm for Optimization Join Order in Sales Transaction System

R.Divika Rathi
Dept. of Computer Science
Bina Nusantara University
Jakarta, Indonesia
rdivika.rathi@binus.ac.id

Keisa Putri Alisa
Dept. of Computer Science
Bina Nusantara University
Jakarta, Indonesia
keisa.alisa@binus.ac.id

Sri Mulyani
Dept. of Computer Science
Bina Nusantara University
Jakarta, Indonesia
sri.mulyani003@binus.ac.id

Dr. Alfi Yusrotis Zakkiyah, S.Pd., M.Si.
Dept. of Computer Science
Bina Nusantara University
Jakarta, Indonesia
alfi.zakkiyah@binus.ac.id

Ayu Maulina, S.Kom., M.Kom.
Dept. of Computer Science
Bina Nusantara University
Jakarta, Indonesia
ayu.maulina@binus.ac.id

Abstract—Query optimization in relational database systems becomes harder with the increase of the volume of data and the complexity of queries, especially when multiple tables are joined. This study proposes a lightweight framework that combines Linear Regression and Greedy Algorithm to optimize join order of hash join queries in a sales transaction system. The dataset consists of 720 instances of query executions on the Classicmodels database. Instances were created by enumerating valid join combinations of two to six tables, with eight filter selectivity levels. Seventeen structural query features were extracted, such as binary table participation indicators, leading table indicators, and filter selectivity. The linear regression model on log1p transformed execution times was able to reach an R2 score of 87.45 %, RMSE of 0.3044 ms and a predicted-to-actual correlation of 0.864. Then, the Greedy Algorithm selected join orders with the lowest predicted cost. It improved execution time in 60.2% of evaluated scenarios with an average speedup of 2.87% and a maximum speedup of 39.62%. These results show that structural query features can be used to guide join order selection without the need for internal DBMS statistics or complex model architectures, providing a viable external optimization layer for small to medium scale database systems.

Keywords—query optimization, join order optimization, linear regression, greedy algorithm, cost estimation, database management system

I. INTRODUCTION

The development of the business world today is growing rapidly along with the increasing use of digital technology in various company activities. Sales transactions, customer management, inventory handling, and business analysis are now carried out digitally and generate large amounts of data. This condition makes the use of Database Management Systems (DBMS) very important to support data storage, management, and processing so that business activities can run more efficiently.

As data volume and transaction complexity continue to increase, query performance has become one of the factors that affects database performance. One of the most influential processes is the join operation, especially when queries involve multiple tables. Choosing an improper join order can increase execution time and computational cost. In addition, the number of possible query plans grows as the number of

tables involved in a query increases, making query optimization more challenging [1].

To address this issue, various query optimization methods have been developed, including cost-based optimization, heuristic approaches, and machine learning-based methods. Several studies have shown that machine learning can help optimizers estimate execution costs and select better query plans [2]. In addition, hash join is widely used as a join execution method in modern DBMSs because it is capable of handling large-scale data processing efficiently [5].

Previous studies have discussed query cost estimation using workload and execution cost models [3], as well as the use of reinforcement learning for join order optimization [4]. However, most of these approaches rely on relatively complex models that require high computational resources.

Based on these issues, this study proposes the use of Linear Regression and Greedy Algorithm to support join order optimization in hash join queries. Linear Regression is used to estimate execution cost based on query features such as join order sequence, number of joined tables, and execution time (ms), with logarithmic transformation applied to stabilize the target variable distribution. Furthermore, the Greedy Algorithm is used to select the query plan with the lowest estimated cost. This approach is expected to provide a lighter optimization process while still producing execution cost estimates that are close to the actual execution time.

II. THEORETICAL BACKGROUND

A. Query Optimization in DBMS

Query optimization is the process of finding the best execution plan for executing a SQL query on a DBMS. The main purpose of query optimization is to reduce the cost of execution, query latency, memory usage and other system resources [6]. In relational databases, the optimizer usually selects an execution strategy according to join order, access path, index selection, and join algorithm.

One of the major problems in query optimization is the exponential increase in the number of possible execution plans with the number of tables in the query. The exponential complexity of finding the optimal join order forces the

optimizer to use heuristic strategies or cost estimation to reduce the search space [7]. Therefore, good search techniques and accurate cost estimation are important to find the optimal execution plans without enumerating all the join orders.

B. Join Processing

A hash join is one of the join algorithms used to process equijoins in relational databases [8]. This algorithm works by building a hash table from the smaller table, then probing the other table to find matching data pairs. Hash Join is considered efficient for large-scale join operations because it reduces the need for repeated scans compared to nested loop joins [8]. The efficiency of Hash Join is greatly influenced by table size, cardinality, and the join order used during query execution. In this study, the join order generated by the Greedy Algorithm is sent to MySQL as an explicit join order, where MySQL automatically uses Hash Join as the execution method for equi-joins between tables. Thus, the optimization focus in this study lies in determining the optimal join order, while the join execution mechanism is fully delegated to the DBMS.

C. Linear Regression

Linear Regression is a supervised learning method that predicts a continuous output variable as a linear weighted combination of input features [9]. Linear Regression is a popular baseline model for prediction and cost estimation task because of its simplicity, low computational cost and high interpretability [10]. In query optimization, Linear Regression can be used as a lightweight cost model to predict query execution time based on structural query characteristics [11]. Given a feature vector $x = (x_1, x_2, \dots, x_n)$, the model outputs a prediction

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n \quad (1)$$

where $\hat{y}$ is the predicted execution cost, x_n is a structural query feature and β_n is the regression coefficient learned during training. For this work, the model is provided with 17 input features describing table participation (binary presence indicators per table), join starting position and filter selectivity. Since execution time is right-skewed, a log1p transformation is applied to the target variable before training to reduce skewness and stabilize variance.

D. Greedy Algorithm

The Greedy Algorithm is a heuristic search strategy that does not guarantee a globally optimal solution, yet it offers a considerable improvement in computational efficiency, particularly for problems with exponentially increasing search space with respect to input size [10]. In this work the algorithm works on a set of valid join permutations of relationally connected tables. The predicted execution cost from the Linear Regression model is used to evaluate each candidate join order and the ordering with the minimum predicted cost is selected as the optimized ordering:

$$J^* = \arg \min_{J_i \in S} \hat{C}(J_i) \quad (2)$$

where J^* is the selected join order and $\hat{C}(J_i)$ is the execution cost estimated by the Linear Regression model. The worst case ordering is the permutation with the highest predicted cost and is used as the baseline for comparison. In contrast to the dynamic programming approaches which need

to store and compare all intermediate subplan costs[8], this strategy runs with much lower memory requirements, which makes it feasible to deploy as an external optimization layer without modifying the DBMS[12]. The optimization process can find good join orders not only depending on the traditional rule-based cost estimators, but combining the cost estimation and heuristic selection. This is in line with the general trend of using machine learning techniques for query optimization.

E. Machine Learning-Assisted Query Optimization

Machine learning-based query optimization uses the history of query executions to guide the optimizer to a more efficient execution plan [6]. Recent work has demonstrated that learned optimizers can significantly improve query execution performance [8], [13]. However, most approaches are based on complex models like deep learning and reinforcement learning which demand high computational resources [4], [14]. Therefore, this study proposes a lightweight approach based on Linear Regression and Greedy Algorithm, which is simpler, more interpretable and more practical for small to medium-size databases. However, most existing machine learning-based approaches are designed for large-scale deployments with extensive training data and tight DBMS integration, which limits their applicability in small to medium-scale database settings [14], [16]. Motivated by these limitations, this study proposes a lightweight framework that relies solely on structural query features, such as table participation, join position, and filter selectivity, combining Linear Regression and a Greedy selection strategy to maintain low computational overhead and high interpretability.

III. METHODOLOGY

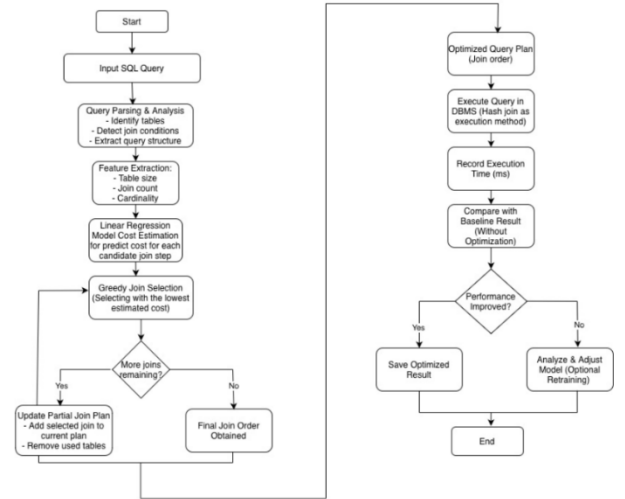


Fig. 1. Flowchart

A. Dataset Description

The dataset is based on the MySQL relational database [17] with 8 inter-related tables. It is designed to support relational operations with SQL JOIN queries. The maximum number of tables in one join combination is 6, and the total number of unique valid join paths is 90. The relational structure of these tables underlies the generation of valid combinations of join queries, during the data acquisition phase.

B. Data Acquisition & Collection

Several SQL JOIN queries with different join order and complexity are generated and executed on MySQL database. Each unique join path is expanded across eight filter selectivity values (0.15, 0.30, 0.45, 0.55, 0.65, 0.75, 0.85 and 0.95) for a total of 720 query execution instances. The query execution is done using a Hash Join based approach following the hash join execution behavior with explicit join ordering. The query profiling is done to collect performance metrics such as execution time. Every execution is saved in raw form for further processing and feature extraction for the machine learning pipeline.

C. Data Processing

The feature extraction process transforms raw SQL query execution results into structured machine learning features. The extracted features include binary table presence indicators (`has_{table}`), binary leading table indicators (`start_with_{table}`), and filter selectivity, resulting in a total of 17 numerical features for 720 rows. In the pre-processing step, numerical features are standardized with StandardScaler and trained on a Linear Regression model. The target variable in the machine learning process is the query execution time, which was transformed using logarithmic normalization (`log1p`) to stabilize its distribution. The predicted execution time of the model is stored together with the actual execution time, creating a prediction map of 720 rows, which is used by the Greedy Algorithm to select the join order with the lowest estimated execution cost, as described in the optimization stage.

D. Join Order Optimization

The cost estimator is a linear regression model trained to estimate the cost of query execution for all possible join combinations. At each iteration, the greedy algorithm chooses the permutation with the lowest estimated query cost until all the tables are joined into a single final query plan. Thus, the join order is chosen based on the best local choice at each step, resulting in an execution plan which is more efficient than the traditional baseline query plan, where the baseline is defined as the permutation having the highest predicted query cost among all valid permutations of the same set of tables. Both components are then evaluated in a structured experimental evaluation of their performance.

E. Experimental Evaluation

The experimental evaluation is designed to evaluate the predictive accuracy of the Linear Regression cost model and the efficiency improvements gained through the Greedy Join Order optimization in query execution.

1) *Linear Regression Cost Model Evaluation*: The evaluation metrics used to evaluate the Linear Regression model are RMSE, MAE, and R^2 Score. RMSE is the mean magnitude of prediction errors, MAE measures the average absolute difference between predicted and actual execution time in milliseconds, and R^2 score is the as amount of variance explained by the model. Lower RMSE and higher R^2 values indicate better model performance.

$$R^2 = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3)$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (5)$$

2) *Greedy Join Order Evaluation*: The runtime efficiency of the Greedy Algorithm, not prediction accuracy, is examined. The evaluation is on 128 permutation scenarios based on the 720-row prediction map. Each query is run 7 times in MySQL and the trimmed mean is used to reduce the noise of server fluctuation. The primary metric is Execution Speedup (%) which is the percentage reduction in the actual execution time achieved by the greedy recommended order compared to the baseline. These measures gauge how well the greedy algorithm can improve the performance of queries.

$$Speedup (\%) = \frac{(Baseline Time - Greedy Time)}{Baseline Time} \times 100\% \quad (6)$$

IV. RESULT AND DISCUSSION

The experimental results are analyzed to evaluate the performance and effectiveness of proposed Linear Regression cost model and Greedy Join Order optimization under different evaluation scenarios.

A. Dataset and Feature Extraction

To evaluate the proposed optimization framework, a dataset of join queries was built from the Classicmodels database. The database contains eight relational tables: customers, orders, orderdetails, products, productlines, payments, employees, and offices. A total of 720 query instances are successfully generated by enumerating all valid relational join combinations of two to six tables, and employing eight filter selectivity levels between 0.15 and 0.95.

TABLE I. JOIN PATH DISTRIBUTION

Number of Tables	Number of Paths	Total Rows
2	12	96
3	16	128
4	19	152
5	24	192
6	19	152
Total	90	720

Each query was represented by 17 structural features such as indicators for table participation, the starting table of the join sequence and filter selectivity. The dependent variable, execution time, was measured directly from MySQL. The execution times of all generated queries were in the range of 0.67 ms to 10.62 ms with an average of 3.01 ms (= 0.86 ms).

A correlation analysis between the extracted features and the observed execution times was performed to better understand the factors that influence query execution cost. The orders table ($r = 0.725$) and the customers table ($r = 0.627$) showed the strongest positive correlations with execution time, and the offices table ($r = -0.517$) and the employees table ($r = -0.517$) showed the strongest negative correlations. These results show that table participation is a significant source of execution cost variation, and provides a useful basis for cost prediction.

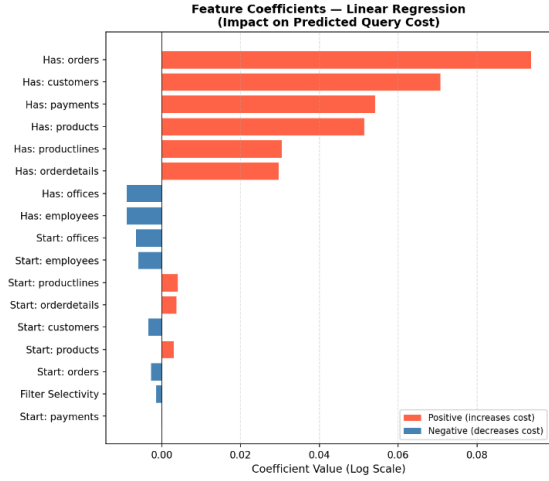


Fig. 2. Feature Coefficients

B. The Linear Regression Model Training

After dataset construction and features extraction, Linear Regression model was trained to estimate join query execution costs. The dataset was divided into training and testing subsets in the ratio of 80:20, resulting in 576 training instances and 144 testing instances. The target variable was $\log_{10}$ transformed for training and all numerical features were standardized using StandardScaler.

TABLE II. LINEAR REGRESSION RESULTS

Metric	Value
R^2 Score	0.8745 (87.45%)
RMSE	0.3044 ms
MAE	0.2206 ms
Number of features	17
Training data	576 rows
Test data	144 rows
Correlations between predicted and actual values	0.864

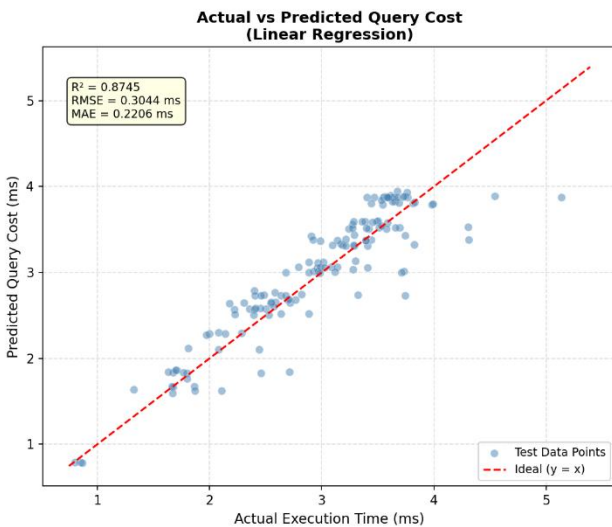


Fig. 3. Linear Regression: Actual vs Predicted

The performance of Linear Regression model is shown in Table II. The model achieved an R^2 score of 87.45% when

only structural query features were used, which indicates that the extracted features can explain a large portion of the variation in the execution time. The RMSE and MAE values of 0.3044 ms and 0.2206 ms indicate that prediction errors were, on average, still below one millisecond.

Furthermore, the correlation coefficient of 0.864 between the predicted and actual values shows that the model preserved the relative ranking of query costs. These results demonstrate that the extracted features hold sufficient information for execution cost estimation and can be employed to support join-order optimization.

C. Greedy Algorithm for Optimization of JOIN Order

The Greedy Algorithm works after the cost estimation phase, choosing join sequences by the execution costs predicted by the Linear Regression model. For each group of identical tables, the algorithm selected the JOIN sequence with the lowest predicted cost and compared it to the JOIN sequence with the highest predicted cost as a baseline.

Each query was executed 7 times in MySQL and took a trimmed mean discarding the highest and lowest execution time to reduce measurement noise. A total of 128 join-order scenarios with at least two alternative valid permutations were analyzed.

TABLE III. GREEDY ALGORITHM RESULTS

Metric	Value
Total scenarios evaluated	128
Greedy reordered JOINS	128 (100%)
IMPROVED (execution time decreased)	77 (60.2%)
DEGRADED (execution time increased)	51 (39.8%)
Average Prediction Cost Improvement (LR)	1.93%
Average Execution Speedup (actual)	2.87%
Maximum Speedup	39.62%
Minimum Speedup (worst)	-26.74%

In 77 out of the 128 evaluated scenarios, the Greedy Algorithm reduced the execution time, which corresponds to 60.2% of the tested cases. The average execution speedup was 2.87%, and the maximum improvement observed was 39.62%.

The best improvement was achieved by changing the JOIN sequence productlines-products-orderdetails-orders-customers to orders-orderdetails-products-productlines-customers. The result indicates that the same set of tables may be involved in different join orders and lead to different execution times.

The results are grouped according to the number of joined tables in order to study the optimization behavior in more detail (see Table IV). The maximum average speedup was achieved for five-table join queries (9.21%), while six-table join queries had an average speedup of -1.39%. These results indicate that optimization efficiency depends on query complexity and join configuration.

TABLE IV. RESULTS BY PATH COUNT

Number of Tables	Scenario	IMPROVED	Avg Speedup	Avg Predicted Cost Improvement
2	48	26 (54.2%)	1.51%	1.14%
3	32	18 (56.3%)	0.98%	1.96%
4	24	19 (79.2%)	5.28%	2.24%
5	16	9 (56.3%)	9.21%	2.86%
6	8	5 (62.5%)	-1.39%	3.76%

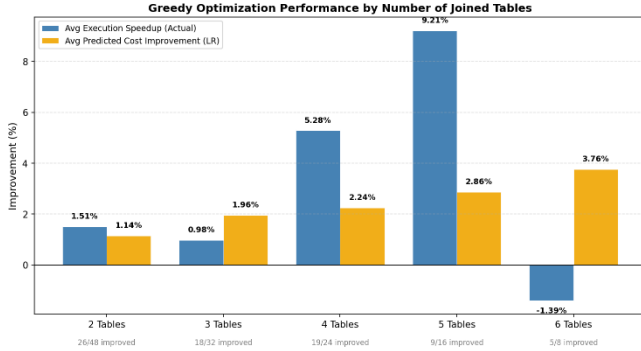


Fig. 4. Greedy Result: Actual vs Predicted

D. Discussion

The results show that the proposed framework can support join-order optimization based on structural query features only. The Linear Regression model was able to predict the costs with an R^2 score of 87.45% and a correlation between the predicted and the actual costs of 0.864. These results show that the model is able to distinguish between relatively expensive and inexpensive join orders, and therefore it can be used to guide the optimization process. The Greedy Algorithm improved the execution time in 60.2% of the evaluated scenarios, with an average speedup of 2.87%. The improvement was modest, but was achieved without modifying the DBMS optimizer, or using internal execution statistics. This result indicates that information about the structure of the query can be useful by itself to guide join order selection.

However, the performance degradation occurred in 39.8% of scenarios. The main reason for this limitation is the errors in the prediction of the Linear Regression model. The greedy algorithm only considers the predicted costs, an incorrect prediction may lead to a bad choice of join-order. The effect is more pronounced for larger join configurations and as the query complexity increases.

Correlation analysis in the evaluated environment showed that table participation and join order are more important factors than filter selectivity in determining execution cost. Larger and more frequently joined tables had more significant relationships with execution time, indicating that table characteristics have an role in determining join performance.

There are several limitations to note. The experiments were only conducted on the Classicmodels database and the model used only structural query features and did not use any DBMS statistics, execution plans or runtime information. Besides, Linear Regression may not be able to capture complex non-linear relations in larger join configurations.

In future work, more expressive prediction models and more query features can be explored to improve the accuracy of cost estimation. Furthermore, the framework can be tested on larger benchmark datasets and on different DBMS platforms to assess its generalizability.

In summary, our framework shows that a light-weight combination of Linear Regression and Greedy-based selection can serve as feasible external optimization layer join-order optimization with low computational overhead, which is suitable for use as an external optimization layer for query processing.

REFERENCES

- [1] B. Warnke, K. Martens, T. Winker, S. Groppe, J. Groppe, P. Adhiyaman, S. Srinivasan, and S. Krishnakumar, "ReJOOSp: Reinforcement Learning for Join Order Optimization in SPARQL," *Big Data Cogn. Comput.*, vol. 8, no. 7, p. 71, Jun. 2024. doi: 10.3390/bdcc8070071.
- [2] N. Sassi and W. Jaziri, "Efficient AI-Driven Query Optimization in Large-Scale Databases: A Reinforcement Learning and Graph-Based Approach," *Mathematics*, vol. 13, no. 11, p. 1700, May 2025. doi: 10.3390/math13111700.
- [3] A. Gounaris, Z. Karampaglis, A. Naskos, and Y. Manolopoulos, "A bi-objective cost model for optimizing database queries in a multi-cloud environment," *J. Innov. Digit. Ecosyst.*, vol. 1, no. 1–2, pp. 12–25, 2014. doi: 10.1016/j.jides.2015.02.002.
- [4] C. Liu, A. Kamali, V. Kantere, C. Zuzarte, and V. Corvinelli, "A novel framework using deep reinforcement learning for join order selection," *arXiv:2412.10253*, Dec. 2024. [Online]. Available: <https://arxiv.org/abs/2412.10253>
- [5] Y. Liu, Z. He, and X. W. Meng, "Accelerating hash join performance by exploiting data distribution," *Int. J. Comput. Trends Technol.*, vol. 56, no. 1, pp. 6–20, 2018. doi: 10.14445/22312803/IJCTT-V56P102.
- [6] H. Lan, Z. Bao, and Y. Peng, "A survey on advancing the DBMS query optimizer: Cardinality estimation, cost model, and plan enumeration," *Data Sci. Eng.*, vol. 6, no. 1, pp. 86–101, Mar. 2021. doi: 10.1007/s41019-020-00149-7.
- [7] A. Hertzschuch, C. Hartmann, D. Habich, and W. Lehner, "Simplicity done right for join ordering," in *Proc. 11th Conf. Innovative Data Systems Research (CIDR)*, 2021. [Online]. Available: https://cidrdb.org/cidr2021/papers/cidr2021_paper01.pdf
- [8] I. Haffner and J. Dittrich, "Efficiently computing join orders with heuristic search," *Proc. ACM Manag. Data*, vol. 1, no. 1, Art. 73, May 2023. doi: 10.1145/3588927.
- [9] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: With Applications in R*, 2nd ed. New York, NY, USA: Springer, 2021.
- [10] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [11] U. Pathak and A. Mankodi, "Redefining cost estimation in database systems: The role of execution plan features and machine learning," *arXiv:2510.05612*, Oct. 2025. [Online]. Available: <https://arxiv.org/abs/2510.05612>
- [12] C. Anneser, N. Tatbul, D. Cohen, Z. Xu, P. Pandian, N. Laptev, and R. Marcus, "AutoSteer: Learned query optimization for any SQL database," *Proc. VLDB Endow.*, vol. 16, no. 12, pp. 3515–3527, 2023. doi: 10.14778/3611540.3611544.
- [13] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making learned query optimization practical," in *Proc. 2021 Int. Conf. Management of Data (SIGMOD '21)*, 2021, pp. 1275–1288. doi: 10.1145/3448016.3452838.
- [14] J. Chen, G. Ye, Y. Zhao, S. Liu, L. Deng, X. Chen, R. Zhou, and K. Zheng, "Efficient join order selection learning with graph-based representation," in *Proc. 28th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD '22)*, 2022, pp. 97–107. doi: 10.1145/3534678.3539303.
- [15] J. Yang, S. Wu, D. Zhang, J. Dai, F. Li, and G. Chen, "Rethinking learned cost models: Why start from scratch?" *Proc. ACM Manag. Data*, vol. 1, no. 4, Art. 255, Dec. 2023. doi: 10.1145/3626769.
- [16] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "Still asking: How good are query optimizers, really?" *Proc.*

VLDB Endow., vol. 18, no. 12, pp. 5531–5536, 2025. doi: 10.14778/3750601.3760521.

- [17] MySQL Tutorial, "MySQL Sample Database," [mysqltutorial.org](https://www.mysqltutorial.org/). [Online]. Available: <https://www.mysqltutorial.org/mysql-sample-database.aspx>. [Accessed: Jun. 10, 2026].